Operating Systems Exam 1 2025/03/12

Please answer questions in order and leave blank for unanswered questions

There are 31 questions

1. (3%) How many times does the following code print “hello”?

```
main () {
    if ( fork () <> 0) printf ("hello\n");
    else if ( fork () <> 0) printf ("hello\n");
    printf ("hello\n");
}
```

Ans. 5

2. (3%) How many times does this code print "hello"?

```
main () {
    int  i;
    for ( i =0; i < 3; i ++) {
        fork();
        execl ("/ bin/echo", "echo", "hello", 0);
    }
}
```

Ans. 2

The execl overwrites the current process by loading the program /bin/echo. The for loop is gone!

3. (4%) Please list two advantages of multi thread processes over single thread processes?

Ans.

a. Creating threads and switching among threads is more efficient.

b. Some programming is easier since all memory is shared among threads – no need to use messaging or create shared memory segments.

4. (6%) What is busy waiting (spinlock)? How can it lead to priority inversion?

Ans:

a. Busy waiting, also known as a spin lock, is the situation when a thread is looping, continuously checking whether a condition exists that will allow it to continue. It is often used to check if a thread can enter a critical section or to poll a device to see if data is ready (or written/sent).

b. Busy waiting can lead to priority inversion under the following situation. Process A is a low priority process that is in a critical section. Process B is a high priority process that is executing a spin lock trying to enter the critical section. A priority scheduler always lets B run over A since it's a higher-priority process. A never gets to run so it can release its lock on the critical section and allow B to run. The situation is called priority inversion because B is not doing useful work; it is essentially blocked by A from making progress.

5. (4%) What is "local replacement" and "global replacement"?

Ans.

Local replacement: a process can only select a replacement frame from its own set of allocated frames.

Global replacement: a process may select a replacement frame from the set of all memory frames.

6. (3%) Please list three attributes that are not shared among threads in the same process.

Ans. Thread ID, Saved CPU registers, Stack pointer, Program counter, Stack (local variables, temporary variables, return addresses), Signal mask, Priority (scheduling information).

7. (3%) What are the parameters of Multilevel Feedback Queue Scheduling? (List at least three)

Ans.

a. number of queues
b. scheduling algorithms for each queue
c. methods used to determine when to upgrade a process
d. methods used to determine when to demote a process
e. methods used to determine which queue a process will enter when that process needs service

8. (2%) What is thrashing?

Ans. If a process's working set is not a subset of it's resident set then the system may exhibit frequent page swapping between the physical memory and the backing store.

9. (4%) What is the difference between a mode switch and a context switch?

Ans.

Mode switch: change CPU execution mode from one privilege level to another, e.g., user → kernel via a trap or system call.

Context switch: save one process' execution context & restore that of another process.

10. (6%) A paging system uses a 2-level page table with an 85% TLB hit ratio. Assume that the TLB search time is ignorable comparing with the memory access time. What is the effective average memory access time comparing with the memory access time?

Ans. The effective average memory access time is the 1.3 times of the memory access time.

a. TLB hit = no page table lookup is needed, # main memory accesses = 1

b. TLB miss = need to look up address via page table, we have a 2-level table, therefore

(1) look up in top-level table to find base of 2nd level table

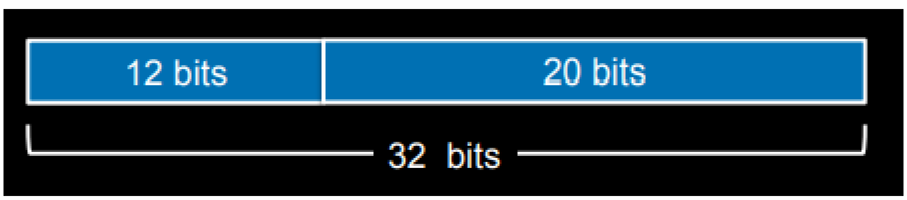
(2) look up in 2nd-level table to find the PTE, giving us the physical frame #

(3) look up main memory ⇒ 3 main memory accesses for a miss

Average memory access time = $(0.85\times1) + (0.15\times3) = 0.85+0.45=1.3$

11. (3%) A system has 32-bit addresses and 1 GB (2^{30}) main memory where each page is 1 MB (2^{20}). How many entries of a page table will contain?

Ans: Each page is 1 MB (2^{20}), so it needs 20 bits as an offset. Thus a 32-bit address can have at most 12-bit (32-20=12) as the page number. Therefore a page table con have at most 2^{12} = 4096 entries.



12.(6%) An inode-based file system uses 8 Kbyte blocks with 4-byte block address. What is the largest file size that the file system can handle if an inode has 12 direct blocks, 1 indirect block, and 1 double indirect block?

Ans: 32 GB + 16 MB + 96 KB

Direct blocks: 12 block pointers × 8K bytes = 96 KB
Indirect block: 1 × 2K block pointers × 8K bytes = 16 MB
Double indirect block: 1 × 2K block pointers × 2K block pointers × 8K bytes = 32 GB
Total size = 32 GB + 16 MB + 96 KB

13.(8%) Given a disk with 200 tracks, numbered from 0 to 199, Assume the disk head is initially located at track 42 and was moving in the direction of increasing track number. Let the requested tracks, in the order received by the disk scheduler, be 116, 22, 3, 11, 75, 185, 100, 87, and 186. What is the order that the requests are serviced by using the Shortest Seek Time First (SSTF), SCAN, C-SCAN, and LOOK scheduling algorithms, respectively?

Ans.

SSTF: 22, 11, 3, 75, 87, 100, 116, 185, 186

SCAN: 75, 87, 100, 116, 185, 186, 22, 11, 3

C-SCAN: 75, 87, 100, 116, 185, 186, 3, 11, 22

LOOK: 75, 87, 100, 116, 185, 186, 22, 11, 3

14.(4%) Please briefly describe the Direct Memory Access (DMA)?

Ans. Direct Memory Access (DMA) is a technique used by computers to transfer data between devices without involving the CPU. DMA allows devices like hard drives, sound cards, and network cards to read or write data directly to or from the main memory, without the need for the CPU to intervene in every transfer, which would waste a significant amount of CPU time. DMA controllers can access the memory independently of the CPU and transfer large amounts of data at high speed, thereby reducing CPU usage and improving system performance.

15. (8%)Please give at least 4 differences between user-level thread and kernel-level thread.

Ans. User-level threads and kernel-level threads are two different approaches to implement threading in an operating system. The key differences between them are as follows:

a. Management: User-level threads are managed entirely by the application, whereas kernel-level threads are managed by the operating system.

b. Scheduling: User-level threads are scheduled by the thread library or the application, while kernel-level threads are scheduled by the operating system's scheduler.

c. Context Switching: Switching between user-level threads is faster because it doesn't require the intervention of the operating system. In contrast, kernel-level threads require a context switch to move from user mode to kernel mode, which can take more time.

d. Scalability: Kernel-level threads can take advantage of multi-core processors and can run concurrently on different cores. In contrast, user-level threads are bound to a single processor, which limits scalability.

16. (3%) What is the race condition?

Ans. A race condition occurs when two or more threads or processes access a shared resource such as a variable, file, or database record, and the final outcome of the program depends on the order in which the threads or processes execute.

17. (6%) What are the internal and external fragmentations?

Ans.

a. Internal fragmentation occurs when a process or program is allocated more memory than it needs, and the unused portion of the memory remains allocated and unusable. This can occur when memory is allocated in fixed-size blocks, and the requested size of the memory allocation is smaller than the block size. The unused portion of the memory in the block is called internal fragmentation.

b. External fragmentation occurs when there is enough total memory available to satisfy a memory allocation request, but the memory is not contiguous or available in a single block. This can happen when memory is allocated and deallocated frequently, leaving small holes or gaps in the memory space that are too small to be used for a new allocation. These gaps can accumulate over time, resulting in a situation where there is enough free memory, but it is scattered in small chunks across the memory space, making it difficult to find a contiguous block of memory to satisfy a large memory allocation request.

18. (2%) A memory management unit (MMU) is responsible for:

(a) Translating a process' memory logical addresses to physical addresses.

(b) Allocating kernel memory.

(c) Allocating system memory to processes.

(d) All of the above.

Ans. (a)

19. (2%) A major problem with the base & limit approach to memory translation is:

(a) It requires the process to occupy contiguous memory locations.

(b) The translation process is time-consuming.

(c) The translation must be done for each memory reference.

(d) A process can easily access memory that belongs to another process.

Ans. (a)

20. (2%) Thrashing in a virtual memory system is caused by:

(a) A process making too many requests for disk I/O.

(b) Multiple processes requesting disk I/O simultaneously.

(c) Processes not having their working set resident in memory.

(d) Slow operating system response to processing a page fault.

Ans. (c)

21. (2%) Which of the following does NOT cause a trap?

(a) A user program divides a number by zero.

(b) The operating system kernel executes a privileged instruction.

(c) A programmable interval timer reaches its specified time.

(d) A user program executes an interrupt instruction.

Ans. (b)

22. (2%) Which state transition is not valid?

(a) Ready → Blocked

(b) Running → Ready

(c) Ready → Running

(d) Running → Blocked

Ans. (a)

23. (2%) Which disk scheduling algorithm is most vulnerable to starvation?

(a) SCAN.

(b) Shortest Seek Time First (SSTF).

(c) LOOK.

(d) First Come, First Served (FCFS)

Ans. (b) With SSTF, outlying blocks may be delayed indefinitely if there's a steady stream of blocks closer to the current head position.

24. (2%) The File Allocation Table of Microsoft's FAT32 file system is a variation of:

(a) linked allocation.

(b) contiguous allocation.

(c) indexed allocation.

(d) combined indexing.

Ans. (a)

25. (2%) Large page sizes increase:

(a) The working set size.

(b) External fragmentation.

(c) The page table size.

(d) Internal fragmentation.

Ans. (d)

26. (2%) A semaphore puts a thread to sleep:

(a) if it tries to decrement the semaphore's value below 0.

(b) if it increments the semaphore's value above 0.

(c) until another thread issues a notify on the semaphore.

(d) until the semaphore's value reaches a specific number.

Ans. (a)

27. (2%) There are three processes on a system with the following performance requirements:
. process A runs every 90 milliseconds, using 30 milliseconds of CPU time.
. process B runs every 60 milliseconds, using 20 milliseconds of CPU time
. process C runs every 500 milliseconds, using 10 milliseconds of CPU time
Use rate-monotonic analysis to assign priorities to the three processes. Assume that priorities range from 0 to 90, with 0 being the highest. Which of the following is a valid set of priority assignments?

(a) PA=40, PB=30, PC=100

(b) PA=50, PB=40, PC=90

(c) PA=70, PB=50, PC=60

(d) PA=40, PB=60, PC=20

Ans. (b)

28. (2%) Which of process state changing only happen in preemptive scheduler but not in non-preemptive scheduler?

(a) running to blocked

(b) blocked to ready

(c) running to ready

(d) ready to running

Ans. (c)

29. (2%) Which type of user thread and kernel thread mapping cannot take advantage of a multiprocessor?

(a) 1:1

(b) N:1

(c) N:M

(d) 1:M

Ans. (b)

30. (2%) Which of the following CPU scheduling algorithms is non-preemptive scheduler?

(a) Shortest Job First (SJF)

(b) Shortest Remaining Time First (SRTF)

(c) Round Robin (RR)

(d) Rate Monotonic

Ans. (a)

31. (2%) Process aging is when:

(a) A long-running process gets pushed to a lower priority level.

(b) A process that did not get to run for a long time gets a higher priority level.

(c) A long-running process gets pushed to a higher priority level.

(d) Memory and other resources are taken away from a process that has run for a long time.

Ans. (b)